

Authentication Flows

What is session?

- When a user logs in, the server creates a unique session for them and sends a special cookie to their browser
- And on each follow-up request the browser sends back the cookies to the server, allowing the server to remember who is making the request
- And we are using ExpressJS and to manage all this things in expressjs, we can use a tool called middleware

Packages we are using

1. express
2. express-session

Basic express-session middleware snippet

```
const app = express()
app.use(session({
  secret: 'a_random_string_that_used_to_sign_and_secure_session_id_cookies',
  // prevents session from being saved on every request if it hasn't been modified.
  resave: false,
  //prevents server from creating sessions for new visitors
  saveUninitialized: true,
  cookie: { secure: true }
}))
```

```

app.post("/login", (req, res) => {
  const { username, password } = req.body;

  if (username || password) {
    console.log("username and password are required");
    res.status(409).send("All the fields are mandatory");
  }

  // we are checking if the username and password are matching or not
  // if yes let them login(or redirectt t) otherwise don't
  if (username === userData.username && password === userData.password) {
    req.session.user = {
      username: username,
    };

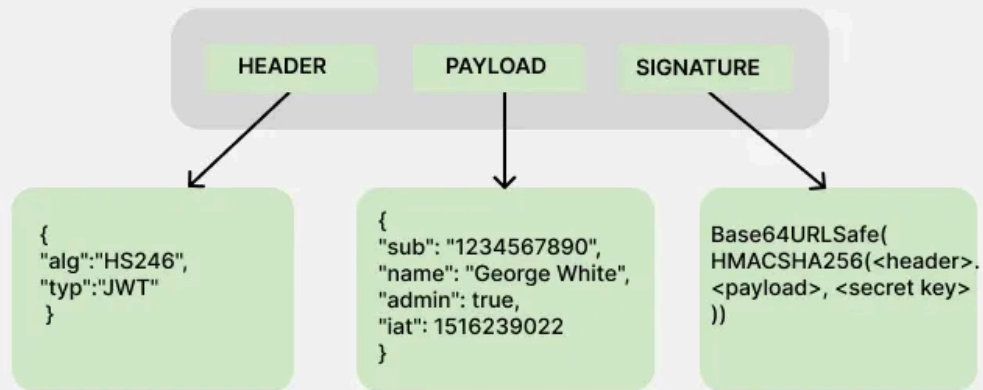
    console.log("Session created: ", req.session);
    res.redirect("/dashboard");
  } else {
    res.send("Invalid username and password");
  }
});

```

- and just because of the middleware we set up, we got an access to a special object `req.session`
- We can attach any data we want to this object, and the middleware will automatically save it and link it to the user's cookie.
- and fun part is that `express-session` **handles the cookie for us automatically.**

JWT

Structure of a JSON Web Token (JWT)



- JWT is used when we want to send information from client to server.
- It is mainly used in web applications and APIs to verify users and prevent unauthorized access.
- A JWT is JSON data secured with a cryptographic signature.

Secure routes and refresh tokens

Authentication Overview

- user authenticate and receives an access token and a refresh token.
- access tokens are short lived
- refresh tokens have longer expiration and stored securely
- when access tokens expire, the client sends the refresh token to get a new access token

1. when the access token expires, the client detects this (after receiving a 401 unauthorized response)
2. so instead of asking the user to login again, the client sends a special request to the refresh token endpoint (client also sends the refresh token to that endpoint for verification purpose)
3. server verifies the refresh token. if it is valid and not expired yet, the server issues a new access token (and maybe the refresh token also)
4. the client receives the new access token and use it for subsequent API requests.
5. This cycle continues until the refresh token expires itself or is revoked

Token Generation and storage

- generate JWT signed with a secret
- store refresh token securely, ideally in an HTTP-only cookie
- we can store the refresh token in a database to allow revocation

Middleware to secure routes

- middleware to verify the access token from request headers
- allow if valid, otherwise deny

Refresh token endpoint

- endpoint that validates the refresh token
 - if valid, issue new access token
 - if invalid/expired, user is logged out
-

Secure Routes + Refresh Tokens

Secure Routes

- Add middleware that checks the presence and validity of JWT token or session cookie.
- For example, in Express:

```
function verifyToken(req, res, next) {  
  const token = req.cookies.accessToken;  
  if (!token) return res.status(401).json({ error: "Unauthorized" });  
  
  try {  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = decoded;  
    next();  
  } catch (err) {  
    return res.status(403).json({ error: "Invalid token" });  
  }  
}
```

Refresh Tokens

- Issue both access and refresh tokens
- store the refresh token in a httpOnly cookie or DB
- we can use it to generate new access token silently

Mini-project: a login/signup system with protected dashboard

Express + JWT

Tech stack we are using

- **express** - HTTP server
- **bcrypt** - password hashing
- **jsonwebtoken** - sign/verify JWTs
- **cookie-parser** - handles cookies (optional for refresh token storage)
- **dotenv** - managing secrets like JWT_SECRET

```
import express, { type Request, type Response } from "express"
import dotenv from "dotenv"
import cookieParser from "cookie-parser"

dotenv.config()

const app = express()
const port = process.env.PORT || 3000

app.use(express.json());
app.use(cookieParser());

app.get('/', (req: Request, res: Response) => {
  res.send('Auth is running...')
})

app.listen(port, () => {
```

```
console.log(`Server running on ${port}`)  
})
```

signup route

1. it should accept name, email, and password
2. hashes the password with bcrypt
3. stores the user in in-memory array

why we hash passwords (instead of storing them as plain text)?

- yes, because password is a very secure and confidential kind of data which needs to be encrypted or make it unrecognizable in order to hide it from external entities

```
app.get('/signup', async (req: Request, res: Response) => {  
  try {  
    const { name, email, password } = req.body  
  
    if (!name || !email || !password) {  
      return res.status(400).json({ message: "All the fields are required" })  
    }  
  
    const existingUser = users.find((user) => { user.email == email })  
  
    if (existingUser) {  
      return res.status(400).json({ message: "User already exists" })  
    }  
  
    // saltrounds shows how many times the hash is internally processed  
    const saltRound = 10;
```

```

const hashedPassword = await bcrypt.hash(password, saltRound)

const newUser: User = {
  id: users.length + 1,
  name,
  email,
  hashedPassword,
}

users.push(newUser)
return res.status(201).json({ message: "User registered successfully", user: { id: newUser.id } })
} catch (error) {
  return res.status(500).json({ message: "Error while registering user!!" })
}
})

```

login route

```

app.post('/login', async (req: Request, res: Response) => {
  const { email, password } = req.body

  // validate input
  if (!email || !password) {
    return res.status(400).json({
      message: "Email and password are required"
    })
  }

  // find user
  const user = users.find((user) => user.email === email);

```

```

    if (!user) {
      return res.status(404).json({
        message: "User not found!!"
      })
    }

    // it basically checks whether the password entered by the user matches
    // with the stored password
    const compare_password = await bcrypt.compare(password, user.hashPassword)

    if (!compare_password) {
      return res.status(401).json({
        message: "Invalid credentials"
      })
    }

    const jwt_secret = process.env.JWT_SECRET;
    if (!jwt_secret) {
      return res.status(401).json({
        message: "JWT_SECRET is not provided"
      })
    }

    // generate JWT
    const token = jwt.sign({ id: user.id, email: user.email }, jwt_secret, { expiresIn: "15m" })

    //send response
    return res.status(200).json({
      message: "Login successful",
      token: token
    })
  })
}

```

Learned these things

- how to compare a plain password with a hash
- how to generate JWT to identify users after login

dashboard route

- Add a JWT Auth middleware and a protected /dashboard route

```
import { type Request, type Response, type NextFunction } from "express";
import jwt from "jsonwebtoken";

const jwt_secret = process.env.JWT_SECRET as string;

export const authenticate = (
  req: Request,
  res: Response,
  next: NextFunction,
) => {
  // Expect: "Authentication: Bearer <token>"

  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res
      .status(401)
      .json({ message: "Missing authentication header" });
  }

  const token = authHeader.split(" ")[1];
  if (!token) {
    return res.status(400).json({
      message: "Token is required",
    });
  }
}
```

```

    }

    try {
      const payload = jwt.verify(token, jwt_secret) as {
        id: number;
        email: string;
        iat?: number;
        exp?: number;
      };

      req.user = { id: payload.id, email: payload.email };
      return next();
    } catch (error) {
      return res.status(403).json({
        message: "Invalid or expired token",
      });
    }
  };
};

```

```

import { authenticate } from "../middleware/auth.js"

app.get('/dashboard', authenticate, (req: Request, res: Response) => {
  // req.user is available and typed (refer to the types.d.ts)
  const user = req.user
  return res.status(200).json({
    message: `Welcome to the dashboard, ${user?.email}`
  })
})

```